5. Architecture & System Diagram

Kalam - Architecture Document

Overview

Kalam is an AI-assisted, deterministically driven welfare scheme matching engine for Indian citizens. It helps users discover government welfare schemes they may be eligible for, through a conversational interface that supports Hindi, Hinglish, and English.

The Golden Rule

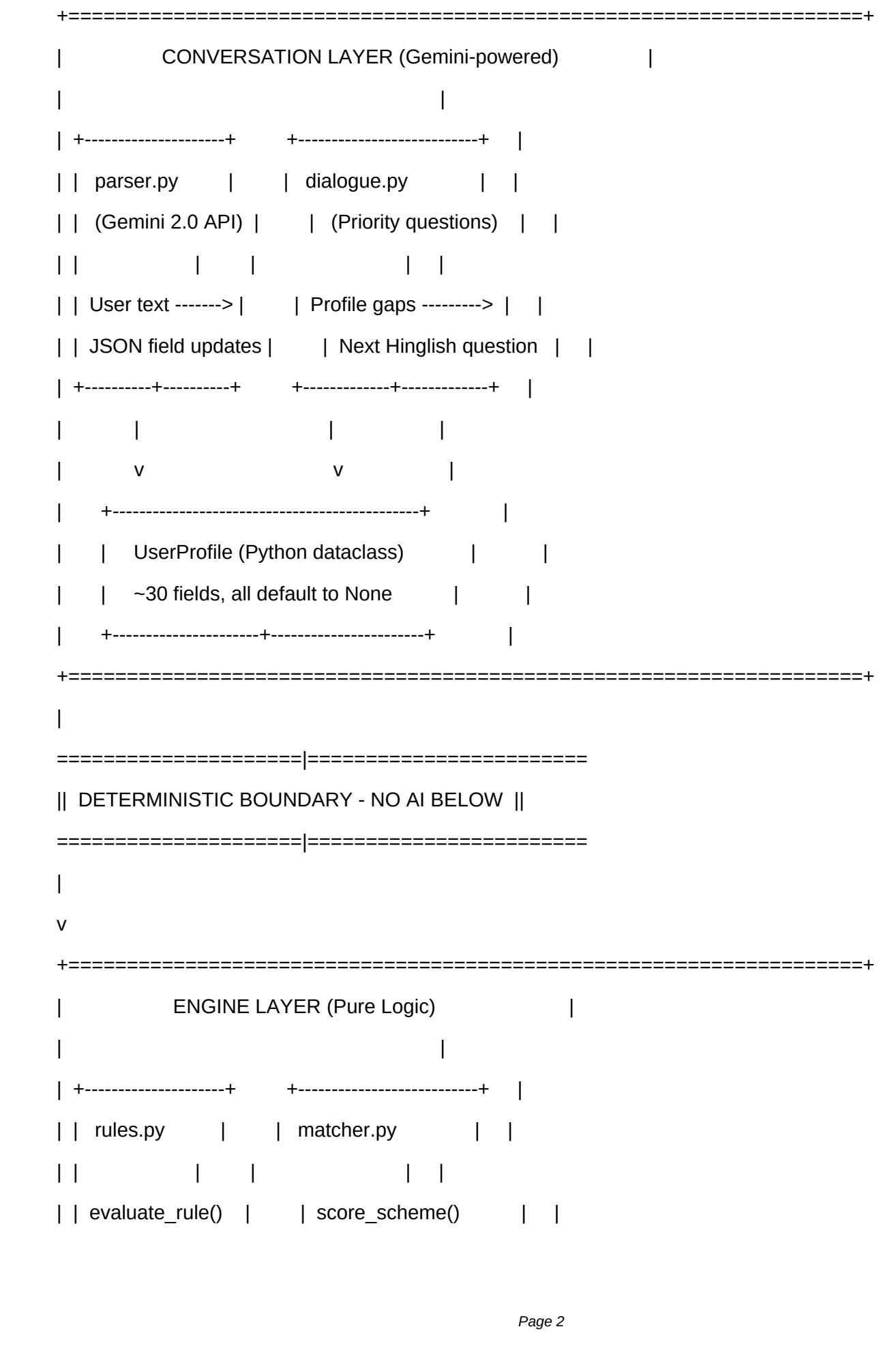
> **The AI (LLM) is ONLY used to read documents and parse conversational user input into structured JSON profiles. The matching logic is 100% deterministic. The LLM never decides eligibility.**

This is the foundational design constraint. If the system doesn't know a value, it flags it as `"unknown"` - it never assumes or hallucinates.

System Architecture Diagram

```
+-------------------------------------------+
|          USER (Chat Interface)            |
|      Hindi / Hinglish / English           |
+--------------------+----------------------+
|
v
```

Kalam Welfare Engine - Official Documentation



```
| | Operators:          |        | Confidence scoring:      |    |
| | gte, lte, eq, in,  |        |   fail -> 0.0            |    |
| | not_in, exists,    |        |   partial -> pass/total  |    |
| | not_exists, between|        |   exact -> 1.0           |    |
| |                    |        | match_all_schemes()      |    |
| +----------+----------+        +-------------+-------------+    |
|            |                          |                |
|            v                          v                |
|      +-----------------------------------------+           |
|      |   Scheme JSON Files (flat-file database)  |           |
|      |   data/schemes/*.json (15 schemes)        |           |
|      +-----------------------------------------+           |
+=================================================================+

+=================================================================+
|                DATA PIPELINE (One-time)                         |
|                                                                 |
|  Raw PDFs --> extract.py --> .txt --> ai_extractor.py --> .json  |
|  (pdfplumber)            (Gemini)        (human review)       |
+=================================================================+
```

Technology Stack

Component	Technology	Purpose
LLM (Parsing)	**Google Gemini 2.0 Flash**	Extract slots from user text, parse scheme documents
LLM SDK	`google-genai`	Official Google AI Python SDK

PDF Extraction	`pdfplumber`	Convert raw PDFs to plain text
Matching Engine	Pure Python	Deterministic rule evaluation, no dependencies
User Interface	`Streamlit`	Conversational chat UI with real-time profile tracking
Data Validation	`pydantic` / `dataclasses`	UserProfile schema with type safety
Language Detection	`langdetect`	Identify Hindi/English/Hinglish input

Key Design Decisions

1. Deterministic Matching vs. LLM-as-Judge

Aspect	Our Approach (Deterministic)	Alternative (LLM-as-Judge)
Eligibility Logic	Pure Python operators (gte, lte, eq, in...)	"Hey Gemini, is this person eligible?"
Reproducibility	Same input = same output, always	May vary with temperature, model version
Auditability	Every decision traceable to rule_id + source_text	Black box
Hallucination Risk	Zero - missing data = "unknown"	High - LLM may infer eligibility
Speed	Microseconds per scheme	Seconds per API call
Cost	Free after profile extraction	$$$ per query

Why this matters: Welfare eligibility is a legal/governmental determination. Citizens deserve reproducible, explainable decisions - not probabilistic guesses from a language model.

2. Google Gemini for NLP (Not for Decisions)

We use **Gemini 2.5 Flash** for exactly two tasks:
1. **Slot extraction** (`parser.py`): Parse conversational Hinglish/Hindi/English text into

structured JSON profile updates
2. **Document parsing** (`ai_extractor.py`): One-time extraction of eligibility rules from scheme PDFs into structured JSON

Gemini was chosen over other models because:
- **Large context window** (1M tokens): Can process large government PDFs in a single call
- **Strong multilingual support**: Native handling of Hindi, Hinglish, and English
- **Cost-effective**: Flash model provides fast, affordable inference
- **Low temperature (0.1)**: Maximizes deterministic extraction behavior

API Key: Set `GEMINI_API_KEY` environment variable.

3. JSON Flat-File Database

We chose flat JSON files over a traditional database because:

- **Transparency:** Non-technical domain experts can read, edit, and validate scheme rules in JSON
- **Version control:** JSON files tracked in Git provide a full audit trail of rule changes
- **Portability:** No database server needed. Works on any machine with Python
- **Human-in-the-loop:** The AI extracts rules as a first pass, but a human reviewer must validate the JSON before production use
- **Scale:** India has ~700+ central schemes. At ~2KB per JSON file, the entire database fits in <2MB

Trade-off: No query optimization, no relational joins. Acceptable because we load all schemes into memory and iterate linearly - which is fast enough for hundreds of schemes.

4. Confidence Scoring Methodology

For each scheme:

```
+--------------------------------------------------+
|  Evaluate every rule against the user profile    |
|                                                  |
|  Rule results:                                   |
|    "pass"      -> rule satisfied                 |
|    "fail"      -> rule NOT satisfied             |
|    "unknown"   -> profile field is None          |
|    "ambiguous" -> rule itself is vague/flagged   |
+--------------------------------------------------+
```

Hard Fail:

If ANY rule -> "fail"

verdict = "ineligible", confidence = 0.0

Exact Match:

If ALL rules -> "pass"

verdict = "eligible", confidence = 1.0

Partial Match:

If mix of pass / unknown / ambiguous (no fails)

verdict = "partial"

confidence = passed_rules / total_rules

gaps = list of missing fields + ambiguous rules

Why not a weighted score? Welfare schemes have binary eligibility. You either meet the criteria or you don't. Weighting would imply some rules matter more than others, which is legally incorrect - all eligibility criteria must be met.

The confidence score represents *completeness of information*, not *likelihood of eligibility*.

A confidence of 60% means "we can verify 60% of the rules; the rest need more data."

Data Pipeline

Raw PDFs (15 scheme documents)

|

v pdfplumber extraction (extract.py)

Extracted Text (data/extracted_text/*.txt)

|

v Gemini AI extraction (ai_extractor.py)

Draft Scheme JSON (data/schemes/*.json)

|

v HUMAN REVIEW (mandatory)

Validated Scheme JSON (15 schemes)

|

v Engine loads at runtime

Deterministic Matching (15 schemes x user profile)

Critical: The AI-extracted JSON is a **draft**. Human domain experts must review and validate before the rules are used for real eligibility determinations.

Scheme Coverage (15 Central Government Schemes)

| # | Scheme ID | Scheme Name | Ministry |

|---|---|---|---|

| 1 | PM-KISAN | PM Kisan Samman Nidhi | Agriculture |

| 2 | PMMY | PM MUDRA Yojana | Finance |

| 3 | MGNREGA | Mahatma Gandhi NREGA | Rural Development |

| 4 | PMAY-G | PM Awaas Yojana - Gramin | Rural Development |

| 5 | PMAY-U | PM Awaas Yojana - Urban | Housing & Urban Affairs |

| 6 | PMUY | PM Ujjwala Yojana | Petroleum & Natural Gas |

| 7 | PMMVY | PM Matru Vandana Yojana | Women & Child Development |

| 8 | DDU-GKY | Deen Dayal Upadhyaya GKY | Rural Development |

| 9 | PM-SVANidhi | PM Street Vendor's Nidhi | Housing & Urban Affairs |

| 10 | NSAP | National Social Assistance (Old Age) | Rural Development |

| 11 | PMSS | PM Scholarship Scheme | Defence |

| 12 | APY | Atal Pension Yojana | Finance |

| 13 | PMEGP | PM Employment Generation | MSME |

| 14 | KCC | Kisan Credit Card | Finance |

| 15 | PMJDY | PM Jan Dhan Yojana | Finance |

| 16 | NSAP-IGNWPS | NSAP Widow Pension | Rural Development |

| 17 | NSAP-IGNDPS | NSAP Disability Pension | Rural Development |

Ambiguity Handling

Real government schemes contain:

- **Definitional ambiguity:** "Small farmer" means different things in different schemes (4 instances documented)

- **Overlap ambiguity:** Two schemes target the same demographic with conflicting criteria (4 instances documented)

- **Contradiction ambiguity:** Age ranges or income thresholds conflict across schemes (2 instances documented)

- **Circular dependencies:** Bank account required for DBT schemes, PMJDY is the scheme to get a bank account (1 instance)

- **Age cliff effects:** Dead zones where no scheme targets a specific age range (1 instance)

Total documented ambiguities: **12** across all schemes (see `data/ambiguity_map.json`)

Our approach:

1. The AI extractor flags ambiguous rules (`"ambiguous": true`)
2. The rule evaluator returns `{"result": "ambiguous"}` for these rules
3. The matcher includes ambiguous results in the **gaps** list, not in the pass/fail count
4. The UI clearly communicates what's ambiguous and why

Module Reference

Module	Purpose	Uses LLM?
`data/extract.py`	PDF -> plain text	No
`data/ai_extractor.py`	Text -> scheme JSON	Yes (Gemini, one-time)
`engine/profile.py`	User profile dataclass	No
`engine/rules.py`	Single rule evaluation	No
`engine/matcher.py`	Multi-scheme scoring	No
`conversation/parser.py`	User text -> profile updates	Yes (Gemini, per message)
`conversation/dialogue.py`	Next question selection	No
`ui/app.py`	Streamlit chat interface	No (calls parser)
`tests/adversarial.py`	10 edge-case profiles	No

Testing Strategy

Adversarial Test Suite (10 profiles)

#	Test Case	Key Validation
1	17-year-old minor	Age boundary: fails at age-gated schemes
2	Remarried widow	Status transition: is_widow=False after remarriage
3	Empty profile	All fields None: every rule returns unknown
4	Govt institutional landholder	Excluded occupation: hard fail
5	Tenant farmer (leases land, owns 0 acres)	Land ownership gate: PM-KISAN fails, other farmer schemes partial
6	BPL widow at age 40	Exact boundary: NSAP-IGNWPS lower limit
7	Urban vendor without Aadhaar	Document gap: multiple schemes fail on has_aadhaar
8	70-year-old disabled person	Multi-scheme overlap: NSAP + IGNDPS
9	Young woman entrepreneur	Loan overlap: PMMY vs PMEGP
10	38-year-old in age dead zone	Age cliff: DDU-GKY expired, APY still open

---

Production-Readiness Gaps

Gap 1: Single-Point-of-Failure on Google Gemini API

Current state: Every user message is routed through `google-genai` for slot extraction. If the API is down, rate-limited, or the free-tier quota (20 req/day) is exhausted, the conversational interface becomes completely non-functional.

Impact: In a production deployment serving thousands of users, a single API outage would brick the entire system. The free tier is wholly inadequate; even a single user session consumes 5-7 API calls.

Mitigation path:
- **Short-term:** Implement an offline regex/heuristic fallback parser (prototype exists in `parser.py` as `_heuristic_extract`) that can handle simple Yes/No/numeric answers without any API call.
- **Long-term:** Deploy a local fine-tuned model (e.g., IndicBERT or a distilled Gemma variant) for slot extraction, eliminating external API dependency entirely. The Gemini API would only be used for the one-time document parsing pipeline.

Gap 2: Stale BPL/SECC Data with No Auto-Update Pipeline

Current state: Six schemes reference "BPL status" which is based on the 2011 Socio-Economic Caste Census (SECC). The BPL list is 15 years old. States have independently updated their criteria, but our rules still use the federal definition.

Impact: A person who is genuinely below the poverty line in 2026 may not appear on the 2011 SECC list, causing false negatives (engine says "ineligible" when they actually qualify). Conversely, the 2011 list includes families who have since risen above the poverty line - causing false positives.

Mitigation path:
- **Short-term:** Flag all BPL-dependent evaluations as `"ambiguous"` (already done in 4 of 6 schemes) and recommend the user verify their name on their state's current BPL list.
- **Long-term:** Build an automated pipeline that scrapes or ingests state-level BPL/SECC updates and hot-reloads the scheme JSONs. This requires partnerships with state IT departments or access to the SECC portal API.

Security & Privacy Notes

- User profiles exist **only in Streamlit session state** (server memory). No persistence.
- Gemini API calls transmit user messages for NLP parsing. In production, consider:
- On-device NLP models for sensitive data
- Data anonymization before API calls
- Compliance with India's DPDP Act 2023
- Scheme JSONs contain no personal data - only government-published eligibility rules.